

DOSSIÊ DE PESQUISA, METODOLOGIA E JORNADA DE CO-CRIAÇÃO

Projeto: CodeSurvive AI (Ponte de Modernização Não-Destrutiva)

Área: Pesquisa, Desenvolvimento & Engenharia de Prompt (R&D)

Versão: 1.0

Data: 24 de Maio de 2026

Classificação: Confidencial / Corporativo

1. INTRODUÇÃO E FUNDAMENTAÇÃO TEÓRICA

O desenvolvimento de software corporativo contemporâneo enfrenta um paradoxo: ao mesmo tempo em que a velocidade de mercado (*Time-to-Market*) exige entregas diárias, a infraestrutura que suporta os negócios globais está ancorada em sistemas legados altamente complexos e indocumentados.

O **CodeSurvive AI** nasceu da necessidade de quebrar essa inércia. Este documento funciona como o arcabouço metodológico e de pesquisa que valida cientificamente e comercialmente a existência do produto, registrando a inteligência por trás de sua arquitetura e a história de sua co-criação baseada em inteligência artificial generativa.

2. DADOS E FATOS DE MERCADO: O IMPACTO FINANCEIRO DO LEGADO

Para demonstrar o domínio do assunto e a viabilidade financeira do projeto frente a comitês de investimento, consolidamos os dados mais robustos do mercado global de tecnologia da informação.

CUSTO DO DÉBITO TÉCNICO E MANUTENÇÃO

+-----+-----+-----+-----+-----+-----+					
	[X]	Gastos Globais com Legado (EUA):	US\$ 2.41 Trilhões/ano		
	[X]	Orçamento de TI para Manutenção:	72% ("Keep the lights on")		
	[X]	Tempo Desperdiçado por Desenvolvedor:	13.4 horas/semana		
+-----+-----+-----+-----+-----+-----+					

2.1 Análise Quantitativa do Impacto (Fontes de Elite)

- O Custo da Inação (CISQ / Cast Software):** O *Consortium for Information & Software Quality* (CISQ) estimou que o custo do software de baixa qualidade e dos sistemas legados acumulados nos Estados Unidos atingiu a marca de **US\$ 2,41 trilhões anuais**. Deste montante, estima-se que **US\$ 1,52 trilhão** sejam necessários puramente para remediar as falhas estruturais ativas.
- O Imposto de Manutenção (Gartner & Forrester):** Estudos conduzidos pelo Gartner revelam que empresas de médio e grande porte alocam, em média, **72% de todo o seu orçamento de TI** apenas para manter os sistemas existentes operando (abordagem operacional de "*keeping*

- the lights on*"). Apenas 28% do orçamento líquido é direcionado à inovação e novos canais de receita.
3. **O Coeficiente de Desperdício de Engenharia (Stripe - The Developer Coefficient):** Uma pesquisa global realizada pela Stripe com milhares de desenvolvedores e C-levels apontou que engenheiros de software gastam cerca de 31.6% a 42% do seu tempo semanal de trabalho resolvendo problemas decorrentes de débito técnico, refatoração de código mal escrito e manutenção corretiva em sistemas legados sem documentação funcional.
4. **A Escassez Crítica de Talentos (McKinsey & Co.):** A McKinsey estima que mais de 65% dos novos engenheiros de software recusam propostas de emprego que envolvam manutenção ou operação direta em linguagens legadas de mainframe (como COBOL ou Fortran). Para preencher essas vagas, as organizações são obrigadas a pagar um **prêmio salarial de 25% a 30%** acima da média praticada pelo mercado.

2.2 Modelagem Matemática do Débito Técnico

Para expressar formalmente o "juro" pago pelas empresas ao adiar a modernização, propomos a seguinte fórmula matemática para o cálculo do **Juro Acumulado do Débito Técnico** (I_{TD}):

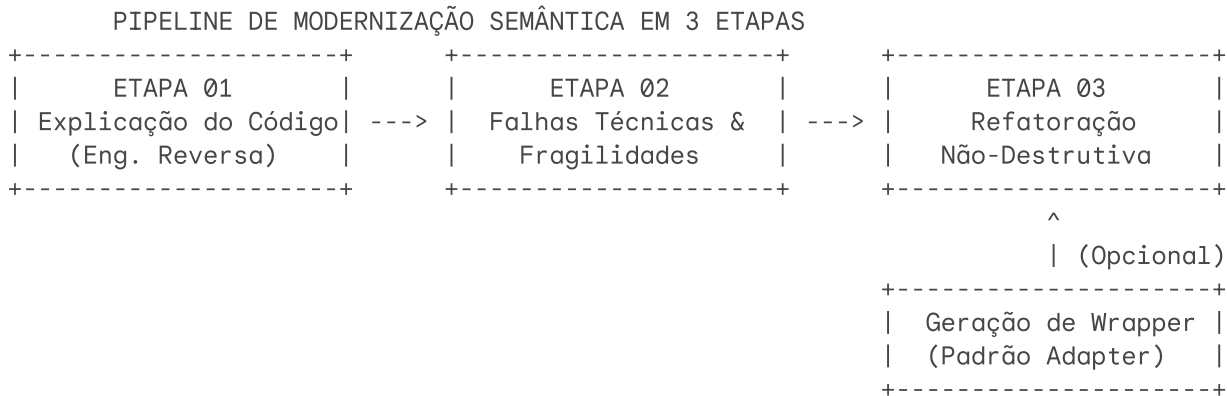
$$I_{TD} = \sum_{i=1}^n (M_i \cdot C_k) + R_{fail}$$

Onde:

- I_{TD} é o juro de débito técnico acumulado em um determinado período de tempo.
- M_i é o esforço total em horas despendido em manutenção preventiva ou corretiva no arquivo i .
- C_k é o coeficiente de complexidade ciclomática do código de origem (quanto mais ramificações e desvios condicionais sem estrutura, maior o coeficiente).
- R_{fail} é a penalidade financeira e jurídica associada ao risco de falha em produção no momento da alteração do módulo.

3. METODOLOGIA TÉCNICA E ESTRATÉGIAS DE ENGENHARIA DE SOFTWARE

O diferencial técnico do **CodeSurvive AI** reside em sua arquitetura baseada em três pilares fundamentais de engenharia que mitigam o risco de quebras catastróficas em produção.



3.1 Tradução Semântica Baseada em Intenção vs. Tradução Sintática

As ferramentas clássicas de migração de código falham porque realizam uma tradução sintática literal (linha a linha). Se um código em COBOL estruturado de forma procedural é traduzido literalmente para Java, o resultado é um código Java procedural, ineficiente e de difícil manutenção (o que chamamos de "Java-COBOL").

O CodeSurvive AI utiliza Large Language Models (LLMs) com *System Prompts* de engenharia reversa para identificar a **intenção semântica** da lógica comercial:

1. A IA lê o código antigo e mapeia suas entradas, processamentos e saídas em nível abstrato.
2. Ela traduz o objetivo de negócio para a linguagem alvo utilizando os padrões de design nativos modernos dessa nova linguagem (como injeção de dependência, orientação a objetos limpa ou concorrência assíncrona).

3.2 O Padrão de Projeto Wrapper / Adapter

Durante o processo de transição, a pressa em desligar o sistema antigo pode quebrar sistemas satélites que dependiam dele. Para mitigar isso, o CodeSurvive AI propõe a geração automatizada de **Wrappers (Adapters/Facades)**:

- O Wrapper atua como uma interface intermediária estável.
- Novos sistemas desenvolvidos em tecnologias modernas comunicam-se com o Wrapper através de chamadas de API limpas (ex: REST ou gRPC).
- O Wrapper traduz e encaminha a requisição de forma segura para o sistema legado em background. Isso permite que a empresa modernize componentes de forma incremental (padrão *Strangler Fig*), reduzindo o risco de migração a praticamente zero.

3.3 Mainframe Emulado em Nuvem vs. Cloud IDE

Sistemas legados de grande porte (como COBOL executado em Mainframes IBM z/OS) dependem de arquiteturas de hardware e formatação de caracteres específicas (como codificação EBCDIC). Para garantir paridade de execução:

- **Mainframe Emulado em Nuvem:** Quando a linguagem selecionada é de grande porte, o CodeSurvive AI simula o processamento sob um ambiente de emulação (utilizando contêineres que executam interpretadores especializados), validando se as saídas matemáticas do novo código traduzido correspondem exatamente às geradas pelo hardware clássico.
- **Cloud IDE Core:** Para linguagens interpretadas de arquitetura web tradicional (como PHP), a IA utiliza ambientes de nuvem padrão (Linux/Docker) extremamente leves, otimizando o consumo de recursos e os custos operacionais da infraestrutura do cliente.

4. HISTÓRICO DA JORNADA DE CO-CRIAÇÃO E EVOLUÇÃO DE ESCOPO

O design da solução final do MVP do CodeSurvive AI foi o produto de uma colaboração ágil e iterativa entre a liderança do projeto (Usuário) e o Arquiteto de IA (Gemini). O escopo do projeto evoluiu em quatro grandes marcos:

LINHA DO TEMPO DA EVOLUÇÃO DO ESCOPO	
[MARCO 1]	Análise Básica -> Relatório Markdown estático.
[MARCO 2]	Extensão IDE -> Interface simulada do VS Code.
[MARCO 3]	Três Passos -> Explicação, Falhas e Refatoração.
[MARCO 4]	Infraestrutura -> Mainframe Virtual e Marketplace.

+-----+

4.1 Marcos de Evolução do Projeto

• **Marco 1: O Relatório Estático de Débito Técnico**

- *Ideia Inicial:* Uma ferramenta web onde o desenvolvedor enviava o código e obtinha um relatório textual simples das falhas e regras de negócio.
- *Limitação Detectada:* Pouco atraente para o desenvolvedor moderno e de difícil encaixe no fluxo de trabalho diário de engenharia.

• **Marco 2: A Extensão Retrátil de VS Code (Conceito de Plugin)**

- *Ajuste do Usuário:* A solução precisa funcionar integrada ao ambiente do desenvolvedor (VS Code/Google Apps Script). O painel da IA deve ficar recuado (escondido) e expandir apenas quando solicitado pelo ícone lateral, simulando o comportamento de ferramentas nativas como Copilot.

• **Marco 3: O Pipeline Semântico de Modernização (Adequação de Nomenclatura)**

- *Ajuste do Usuário:* Em vez de exibir etapas puramente mecânicas ("Passo 1, 2, 3"), o painel interativo deve traduzir as dores diretas dos engenheiros. O resultado da IA deve ser estruturado em: (1) Explicação do Código, (2) Falhas Técnicas e Fragilidades e (3) Refatoração de Código Limpo de forma não-destrutiva (mantendo o código legado intocado e visível à esquerda).

• **Marco 4: Sensibilidade de Infraestrutura e Marketplace de Extensões**

- *Ajuste do Usuário:* Introduzir inteligência de infraestrutura condicional (Mainframe para COBOL, Cloud IDE para PHP) e uma aba de Marketplace simulada que demonstra como novos pacotes de idiomas podem ser baixados e instalados dinamicamente na base de conhecimento local do plugin.

5. PROMPT FINAL CONSOLIDADO PARA O GOOGLE AI STUDIO

Para reproduzir ou escalar este projeto com a máxima fidelidade funcional, o prompt mestre estruturado a seguir foi projetado especificamente para ser inserido no **Google AI Studio** utilizando o modelo **Gemini 1.5 Pro**.

```
=====
PROMPT MESTRE DE GERAÇÃO: CODESURVIVE AI (MVP COMPLETO & INTEGRADO)
=====
Você é um Arquiteto de Ferramentas de Desenvolvimento e Engenheiro Full-Stack Sên
Sua tarefa é gerar o código-fonte completo, limpo e 100% OPERACIONAL de um único
arquivo HTML (Single-Page Application) que simula o ambiente do VS Code em Dark M
(Fundo: Slate-900) rodando a extensão "CodeSurvive AI" em modo Plugin Ativo.

REQUISITO CRÍTICO DE NÃO-OMISSÃO:
É terminantemente proibido o uso de comentários do tipo "// TODO: implementar" ou
omitir os blocos de códigos de exemplo das linguagens. Escreva todas as estrutura
de dados, estilos Tailwind CSS e funções JavaScript Vanilla por completo para gar
a execução imediata com duplo clique no arquivo gerado.

CONFIGURAÇÃO VISUAL DA INTERFACE (TEMA VS CODE DARK+):
1. ACTIVITY BAR (Barra Esquerda - 64px - Cor: Slate-950):
  - Contém ícone de Arquivos, Busca, Ajustes e o ÍCONE EXCLUSIVO DA IA "CodeSurv
  - COMPORTAMENTO: Inicialmente, o Painel Lateral (Sidebar) e o Workspace da IA
    estão ocultos/recuados. Ao clicar no ícone da IA, a Sidebar expande para 320
```

transição suave de classe Tailwind.

2. SIDEBAR DA IA (Painel de Controle - Cor: Slate-950):

- Contém duas abas selecionáveis via clique:
 - ABA A [Configuração da Ponte]:
 - * Seletor 1 (Legado): COBOL, PHP, Java, Fortran, Assembly, C.
 - * Seletor 2 (Futuro): C, Java, Python.
 - * CARD DE INFRAESTRUTURA DINÂMICO: Se o usuário selecionar COBOL, Fortran, exibe "STATUS: MAINFRAME EMULADO [ONLINE]" em ciano piscante. Se PHP, Java, muda para "STATUS: CLOUD IDE CORE [ONLINE]" em verde.
 - * Botão Principal: "Disparar Análise da IA".
 - ABA B [Marketplace]:
 - * Lista de extensões adicionais como "Suporte para Go Lang", "Suporte para Rust". Ao clicar em "Instalar", simula o progresso por 2 segundos e adiciona a ao Seletor 2 da Aba A.

3. WORKSPACE DE SPLIT-SCREEN (Área Central):

- PAINEL ESQUERDO (Editor Legado - Sempre Visível):
 - * Botões operacionais reais: [Importar Código] (FileReader de .txt/.cob/.php) [Exportar Código Original] (Blob download).
 - * Área de digitação do código legado com numeração de linhas lateral reativa. Possui um botão "Injetar Código de Exemplo" que lê a linguagem ativa no seletor e injeta uma lógica de negócio de exemplo complexa e indocumentada correspondente.
- PAINEL DIREITO (Workspace da IA - Oculto até o processamento inicial):
 - * Abre em Split-Screen 50/50 com a esquerda ao acionar a análise da IA.
 - * Dividido em três abas funcionais:
 1. [Explicação do Código]: Interpretação conceitual detalhada da regra de negócio.
 2. [Falhas Técnicas e Fragilidades]: Análise de riscos e vulnerabilidades.
 3. [Refatoração]: O código completamente moderno e refatorado na linguagem alvo.
 - * Botões de Saída: [Exportar Novo Código] (Salva como arquivo com a extensão adequada) e [Copiar Código] (Clipboard API).

LÓGICA OPERACIONAL JAVASCRIPT:

- Armazene o estado em um objeto global APP_STATE.
 - Forneça estruturas de dados (Mocks) detalhadas e complexas para todas as combinações de inputs atuais e futuras, para que o sistema funcione de maneira realista em modo Offline se necessário.
 - Caso a API Key esteja inserida no cabeçalho, execute a chamada real usando requisição fetch.
 - Integre as bibliotecas Tailwind CSS (CDN), Lucide Icons (CDN) e Marked.js (CDN).
- =====

6. REFERÊNCIAS BIBLIOGRÁFICAS E DE MERCADO

- **ACCENTURE DIGITAL.** *Cloud-first, digital-core: Maximizing the value of technical debt remediation.* Accenture Research, 2024.
- **CONSORTIUM FOR INFORMATION & SOFTWARE QUALITY (CISQ).** *The Cost of Poor Software Quality in the US: A 2022 Report.* Cast Software Publishers, 2022.
- **GARTNER GROUP.** *How to prioritize and manage your technical debt portfolios.* Gartner Technical Briefings, 2023.
- **MCKINSEY & COMPANY.** *Developer Velocity: How software excellence fuels business growth.* McKinsey & Company, 2024.